UACAM
Universidad Autónoma de Campeche

2doParcial_Practicas03_ ServidorConcurrente



Concurrencia y paralelismo

Profesor: Sergio Noh Puch
Alumna: Angela Alicia Pimienta Diaz

Introducción

En esta práctica se trabajó con un cliente y un servidor TCP escritos en Java usando las clases Socket y ServerSocket. El código original tenía un problema: la conexión se cerraba después del primer mensaje, por lo que no era posible mantener una conversación. El objetivo fue modificar ambos programas para que el cliente permaneciera conectado, pudiera enviar varios mensajes en una misma sesión y la conexión se terminara de forma ordenada cuando el usuario escribiera la palabra clave “salir”.

Desarrollo:

Qué cambió respecto al código original

En el servidor (ConcurrentServer), el método handleRequest llamaba a readLine() una sola vez. Al recibir el primer mensaje el método terminaba y el try-with-resources cerraba el socket. Lo cambié por un ciclo while ((msj = in.readLine()) != null), de modo que el servidor sigue atendiendo al mismo cliente y responde “Recibido: ...” a cada mensaje. También agregué la palabra clave “salir”, ante la cual el servidor se despide y sale del ciclo. Además, una desconexión abrupta de un cliente ahora solo se registra en consola y ya no lanza una RuntimeException.

En el cliente (ConcurrentClient) conservé el ciclo de lectura del teclado para enviar múltiples mensajes. El cambio principal fue quitar el System.exit(0) del hilo lector durante la salida normal y reemplazarlo por un cierre ordenado.

Desarrollo:

Por qué la conexión se mantiene viva

El hilo del servidor que atiende al cliente no llega al final del método mientras el cliente siga conectado, porque readLine() es una operación bloqueante: si no llegan datos, el hilo simplemente espera sin consumir CPU. Como cada cliente tiene su propio hilo, esa espera no impide atender a otros clientes.

Mecanismo de salida limpia

Cuando el usuario escribe "salir", el cierre ocurre en estos pasos:

1. El cliente envía "salir" al servidor y rompe su ciclo de teclado

1.El cliente llama a socket.shutdownOutput() para avisar que no enviará más datos (fin de flujo).

2.El servidor detecta "salir", responde con una despedida y sale de su ciclo. El try-with-resources cierra los flujos y el socket, y el hilo del cliente termina.

3.El cliente recibe la despedida; su hilo lector lee el fin de flujo (null) y termina. El cliente espera ese fin con lector.join(3000), y si el servidor no respondiera, fuerza el cierre del socket.

4.El try-with-resources del cliente cierra el socket y el programa termina por sí solo, sin System.exit.

Para comprobarlo, después de una sesión revisé los hilos del servidor con jstack y no quedó ningún hilo de cliente activo. También probé un cliente que se cerró sin escribir "salir": el servidor lo liberó correctamente y siguió funcionando.

Si el cliente solo guarda silencio, el servidor continúa esperando en readLine(). Si el cliente cierra su lado de la conexión (porque escribió "salir" o terminó el programa), readLine() devuelve null; el ciclo termina, se cierran los flujos y el socket, y el hilo finaliza sin dejar recursos colgados.

Conclusión

Con estas modificaciones se logró que el cliente y el servidor mantuvieran una sesión persistente, en la que se pueden intercambiar varios mensajes por la misma conexión. Entendí que el problema original no estaba en el protocolo, sino en que el servidor leía una sola vez antes de cerrar el socket, y que un ciclo de lectura bloqueante es lo que sostiene la conversación. También comprobé la importancia de cerrar los recursos de manera ordenada: usar try-with-resources, avisar el fin del envío con shutdownOutput() y esperar la terminación del hilo lector evita hilos y sockets colgados, y permite que el servidor siga atendiendo a otros clientes sin verse afectado.

Evidencias





